

# Reinforcement Learning

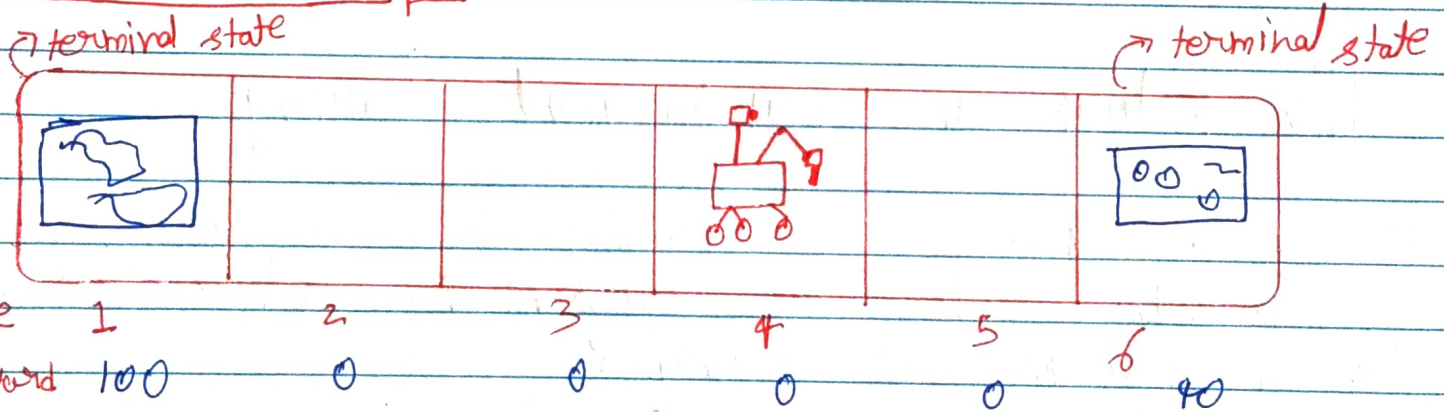
↳ what is RL?

↳ it works on incentivising the model to learn when the model does something good we give it a +ve reward when a model does something bad we give it a -ve reward.

↳ Applications

- ↳ Controlling robots
- ↳ Factory optimization
- ↳ Financial (stock) trading
- ↳ Playing games (including video games)

## Mars Rover example



→ Let's say you have a crazy terrain at state **1** & a less exciting terrain at state **6**.

So you should incentivize the model ~~that~~ model higher for being at state "1" than state "6" or any other state.

**Terminal state** ∴ The state after which no actions are taken

model  $\rightarrow$

$(S, a, R(S), S')$

$S \rightarrow$  initial state

$a \rightarrow$  action

$R(S) \rightarrow$  Reward at being on a state

$S' \rightarrow$  The new state

ex  $\Rightarrow$

$(4, \leftarrow, 0, 3)$

Here  $\rightarrow$  the initial state at which the rover is, is 4.

$\rightarrow$  it takes an action moving towards left

$\rightarrow$  it gets zero reward

$\rightarrow$  The new state of the rover is 3

above  $R(S)$  means  $R$  of  $S$  the reward is associated to the state of the model

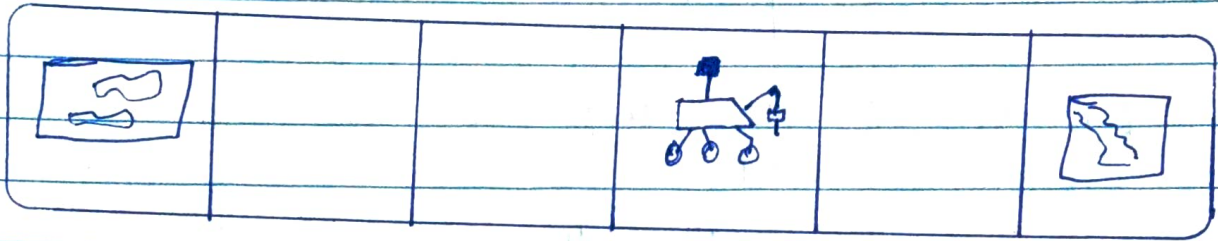
## The "Return" in reinforcement learning

The "Return" answers the question, how do you know if a particular set of rewards is better or worse than a different set of rewards?

imagine if you have a 5 dollar bill at your feet which you can reach out & pick up & you can take a walk for 30 min to another part of the town where you will find a 10 dollar bill, which one are you choosing?

$\left. \begin{array}{l} \text{\$10 is better than \$5 but you will still have to} \\ \text{walk 30 min} \end{array} \right\}$

Return is calculated by as the sum of the rewards by but weighted by one additional factor which is gamma " $\gamma$ ".



|               |     |   |   |   |   |    |
|---------------|-----|---|---|---|---|----|
| <u>State</u>  | 1   | 2 | 3 | 4 | 5 | 6  |
| <u>Reward</u> | 100 | 0 | 0 | 0 | 0 | 40 |

$$\text{Return} = 0 + (0.9) \times 0 + (0.9)^2 \times 0 + (0.9)^3 \times 100 = 0.729 \times 100 = \boxed{72.9}$$

$$\text{Return} = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 \text{ ---- (until terminal state)}$$

$$\text{Discount factor} \rightarrow \gamma = 0.9, 0.99$$

$$\gamma = 0.5$$

Example, let's say your rover is at state 4, then

You can calculate the return for both state 1 & state 6 & whichever one has the highest return we will choose or move towards that state,

$$\gamma = 0.5$$

$$\text{return for state 1} = 0 + (0.5)(0) + (0.5)^2(0) + (0.5)^3(100) = 12.5$$

$$\text{return for state 6} = 0 + (0.5)(0) + (0.5)^2(40) = 10$$

Here,

$12.5 > 10$ , so we will move towards state 1.

## Making decisions:- Policy in reinforcement learning

There are many different ways to take action in reinforcement learning problem.

E.g.,  $\rightarrow$  we can always go for the nearest reward

- $\rightarrow$  Can always go for a larger reward
- $\rightarrow$  Can always go for smaller reward
- $\rightarrow$  or you could choose to go for higher reward unless you are just one step away from the lesser reward.

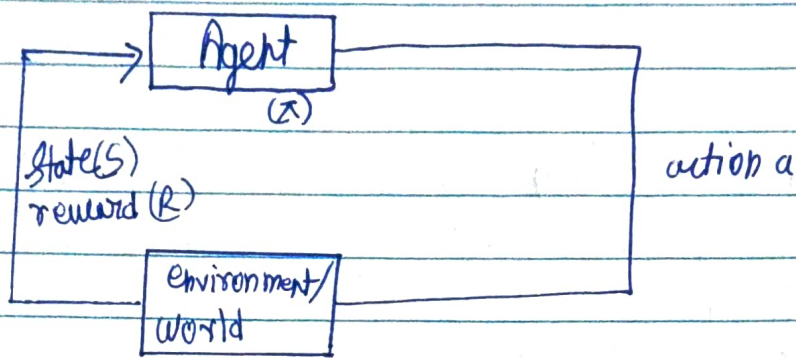
in RL our goal is to come up with a function which is called a policy  $\pi$ .

Policy takes any state "s" as an input & map it to some action "a" that it wants us to take

The goal of RL - find a policy  $\pi$  that tells you what action ( $a = \pi(s)$ ) to take in every state (s) so as to maximize the return.

## Markov decision process (MDP)

$\hookrightarrow$  In a markov decision process the future depends only on where you are now, not on how you got here



MDP

- ↳ calculate action through policy & perform it
- ↳ analyse the environment/world
- ↳ compute state (s) & Reward (R)
- ↳ repeat

**STATE-ACTION** value function also called  $Q$  fn or optimal  $Q$  function denoted by " $Q$ ",

$Q(s, a) =$  Return, if you

- Start in state  $s$ .
- take action  $a$  (once).
- Then behave optimally after that

$C \Rightarrow$

|        |     |    |    |      |    |    |
|--------|-----|----|----|------|----|----|
| return | 100 | 50 | 25 | 12.5 | 20 | 40 |
| action |     | ←  | ←  | ←    | →  |    |
| reward | 100 | 0  | 0  | 0    | 0  | 40 |
| state  | 1   | 2  | 3  | 4    | 5  | 6  |

$$Q(2, \rightarrow) = 0 + (0.5)(0) + (0.5)^2(0) + (0.5)^3 100 = 12.5$$

you go from 2 → 3, then 3 → 2, then 2 → 1

0      0      0      then      100

$$Q(2, \leftarrow) = 0 + (0.5) 100 \Rightarrow 50$$

To pick actions,

→ The best possible return from state  $S$  is  $\max_a Q(S,a)$   
meaning the maximum value of STATE-Action  
ex →  $Q(4, \leftarrow) = 12.5$  ✓  
 $Q(4, \rightarrow) = 10$  ✗

→ The best possible action in state  $S$  is the action  $a$  that gives  $\max_a Q(S,a)$ .

## Bellman Equation

It is used to find the " $Q$ " value, or state-action value function.

↳ it is a recursive function

$S$  = current state,  $R(S)$  = reward of current state

$a$  = Current action,  $S'$  = state you get to after taking action  $a$ .

$a'$  = action that you take in state  $S'$

$$Q(S,a) = R(S) + \gamma \max_{a'} Q(S',a')$$

analogy,

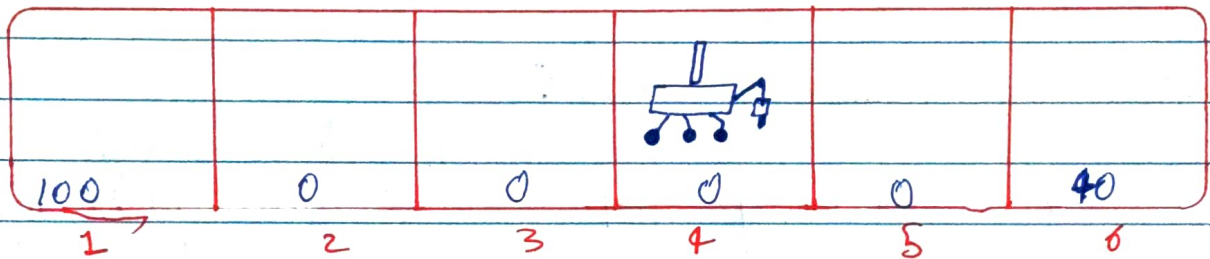
$$Q(S,a) = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \dots$$

$$Q(S,a) = R_1 + \gamma [R_2 + \gamma R_3 + \gamma^2 R_4 + \dots]$$

↳  $Q(S',a')$

## Bellman equation in a stochastic environment

Imagine your rover decides to move to the left but it accidentally moves to the right now what just happened, it was supposed to move to the left, it turns out the surface of it was slippery & there is a 10% chance that the rover might slip, so how you gonna calculate the return state-action value.



So,

the optimal way for the rover to move in this left direction but it might slip with the probability of 0.1 or 0.

So the sequence

might look like

|   |   |    |     |   |     |
|---|---|----|-----|---|-----|
| 0 | 0 | 0  | 100 |   |     |
| 0 | 0 | 0  | 0   | 0 | 100 |
| 0 | 0 | 40 |     |   |     |

in this case just take the average of all the reward sequences,

$$\text{Return} = \text{Average} (R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \dots)$$
$$\text{or} = E(R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \dots)$$

## Expected Return or Average Return

Goal of RL :-

Choose a policy  $\pi(s) = a$  that will tell us what action  $a$  to take in state  $s$  so as to maximize the expected return

Bellman eqn :-

$$Q(s, a) = R(s) + \gamma E \left[ \max_{a'} Q(s', a') \right]$$

## Continuous State Space applications

Discrete vs continuous state :-

In the example of the mass rover we only had 6 discrete states but in reality, we have a vector of variables that can take any no of values & that vector will define the state of the model.

| Ex → | Car                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | helicopter                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S =  | $\begin{bmatrix} x \\ y \\ \theta \\ \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}$ <ul style="list-style-type: none"><li><math>x</math> → Position along horizontal axis</li><li><math>y</math> → Position along vertical axis</li><li><math>\theta</math> → Orientation / heading angle</li><li><math>\dot{x}</math> → Velocity in x-direction</li><li><math>\dot{y}</math> → Velocity in y-direction</li><li><math>\dot{\theta}</math> → Angular velocity / turning rate</li></ul> | $\begin{bmatrix} x \\ y \\ z \\ \phi \\ \theta \\ \psi \\ \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$ <ul style="list-style-type: none"><li><math>x</math> → Position along forward axis</li><li><math>y</math> → Position along left/right axis</li><li><math>z</math> → Position along the vertical axis</li><li><math>\phi</math> → (roll) rotation around x-axis (tilt left/right)</li><li><math>\theta</math> → (pitch) rotation around y-axis (tilt forward/backward)</li><li><math>\psi</math> → (yaw) rotation around z-axis (heading)</li><li><math>\dot{x}, \dot{y}, \dot{z}</math> → Linear velocities (rate of change of positions in <math>(x, y, z)</math>)</li><li><math>\dot{\phi}, \dot{\theta}, \dot{\psi}</math> → Angular velocities (rate of change of Roll, Pitch &amp; Yaw)</li></ul> |

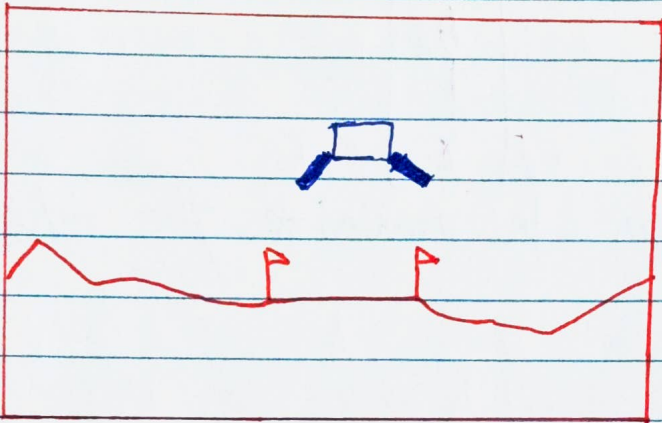
## A basic Lunar Lander example -:

Actions -:

- ① do nothing
- ② left thruster
- ③ main thruster
- ④ right thruster

$S =$

$$\begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \\ \theta \\ \dot{\theta} \\ l \\ r \end{bmatrix}$$



$x \rightarrow$  left or right

$y \rightarrow$  how high or how low

$\dot{x}, \dot{y} \rightarrow$  rate of changes in  $x$  &  $y$

$\theta \rightarrow$  the angle at which the ~~rover~~ of the rover

$\dot{\theta} \rightarrow$  change in the angle of the rover

$l \rightarrow$  a yes/no, is the left leg touching the ground

$r \rightarrow$  a yes/no, is the right leg touching the ground

### Reward function -:

$\rightarrow$  Getting to landing pad : 100 - 190

"it depends upon how well it's flown & gotten to the center"

$\rightarrow$  Additional reward for moving away/toward from pad.

$\rightarrow$  Crash = -100, Soft landing = +100, leg grounded = +10

$\rightarrow$  fire main engine = -0.3, fire side thruster = -0.03

we put a little -ve reward on firing thrusters to prevent the model from over using the thrusters.

Lunar lander problem,

Learn a policy  $\pi$  that, given

$S =$

$$\begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \\ \theta \\ \dot{\theta} \\ 1 \\ \gamma \end{bmatrix}$$

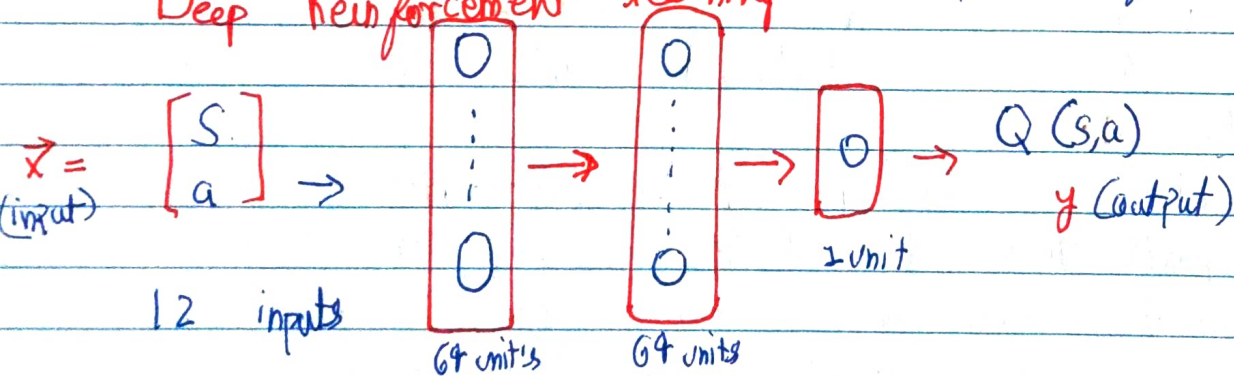
Picks action  $a = \pi(s)$  so as to maximize the return

Discount factor  $(\gamma) = 0.985$

## Learning the state-value function

The key idea is to compute or approximate the state action value function  $Q$  of SA, & that will let us pick good action.

Deep Reinforcement learning



$\Rightarrow$  In a state  $s$ , use neural network to compute,  $Q(s, \text{nothing})$ ,  $Q(s, \text{left})$ ,  $Q(s, \text{main})$ ,  $Q(s, \text{right})$   
Pick the action  $a$  that maximizes  $Q(s,a)$

in particular, for the lunar lander,

the state  $S$  and any action  $A$  & put them together. <sup>we're going to take</sup>

the input data will look like,

Here we will put in all the values of states  $x_1, x_2, \dots$  & we will use one hot encoding for the actions

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} S \\ A \end{bmatrix}$$

whenever you have a state " $s$ " you can compute  $Q(s, A)$  by every action " $a$ " & whichever action gives you the maximum reward you do that.

So,

how do you train a neural network to output  $Q(s, A)$

The approach is to use bellman's equations to create a training set with lot's of examples  $x$  &  $y$ , & then we'll use supervised learning to optimize the model, here  $x$  is state action pairs &  $y$  is  $Q(s, A)$ .

So,

how do you get a training set with values for  $x$  &  $y$  that you can then train a neural network on?

let's look at the Bellman equation, for the answer,

So,

$$Q(s, a) = R(s) + \gamma \max_{a'} Q(s', a')$$

So,

we're going to use the linear learner, & just try different actions in it,

if we don't have a good policy yet, we'll take actions randomly,

by doing this we'll observe at a lot of examples when we are in some state  $s$  & we took some action maybe a good or a bad one, either way, & then we got some rewards  $R(s)$  for being in that state,

& to the result of our action we got to some new state  $(s')$ .

$$\begin{pmatrix} s^{(1)} \\ a^{(1)} \\ R(s^{(1)}) \\ s'^{(1)} \end{pmatrix}$$

$$\vdots$$

$$\begin{pmatrix} s^{(n)} \\ a^{(n)} \\ R(s^{(n)}) \\ s'^{(n)} \end{pmatrix}$$

each of these tuples of ~~into~~ data will ~~are~~ help us create a single  $(x, y)$  pair, by putting them into bellman eqn.

$$x^{(1)} = \begin{bmatrix} s^{(1)} \\ a^{(1)} \end{bmatrix}, \quad y^{(1)} = \left[ R(s^{(1)}) + \gamma \max_{a'} Q(s'^{(1)}, a^{(1)}) \right]$$

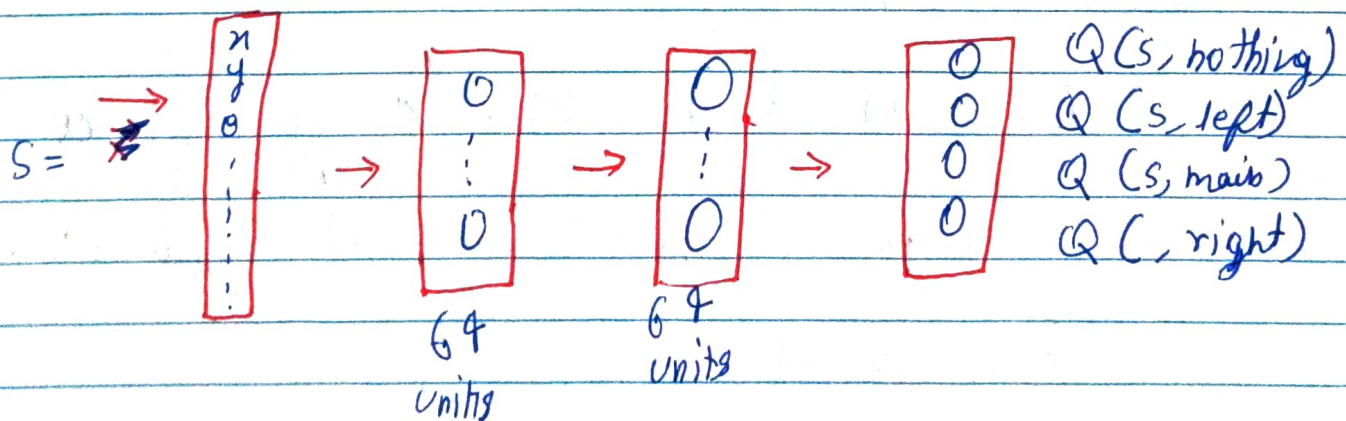
do this & collect like 10,000 training example, it might depend upon your case, but considering lunar lander example 10,000 is fine.

All of what we studied here is called "DQN" Deep Q Network, because you are using deep learning to train a model to learn the Q function

## Algorithm refinement: Improved neural network architecture.

in the previous neural net architecture you had to carry out inference for all the actions that can be performed, so you have to generate inference as ~~the~~ many times as the number of actions.

<sup>Instead</sup> you can train an algorithm that outputs all ~~of~~ the ~~state~~ state action pairs values simultaneously



## Algorithm refinement: $\epsilon$ -greedy Policy.

while you're still learning how to approximate  $Q(s,a)$  you need to make take some actions in the lunar lander.

How do you pick those actions while you're still learning, greedy policy. This issue is resolved by epsilon greedy policy

### Learning Algorithm

Initialize neural network randomly as guess of  $Q(s,a)$ .

Repeat { take actions in the lunar lander. get  $(s,a,R(s),s')$ . Store 10,000 most recent  $(s,a,R(s),s')$  tuples

Train model:

Examples using Create training set of 10,000

$$X = (s,a) \quad \& \quad y = R(s) + \gamma \max_{a'} Q(s,a').$$

Train  $Q_{new}$  such that  $Q_{new}(s,a) \approx y$ .

$$f.w.b \quad Q \approx y$$

$$\text{Set } Q = Q_{new}$$

How to choose actions while still learning?

in some state  $s$

Option 1:

Pick the option  $a$  that maximizes  $Q(s, a)$

Option 2:

Step ① → with probability 0.95, Pick the action  $a$  that maximizes  $Q(s, a)$ ,  
Step ② → with probability 0.05, picks an action  $a$  randomly.

why do we want to pick a random action occasionally  
↳ suppose there's some strange reason that  $Q(s, a)$  was initialized randomly so that the learning algo thinks that firing the main thruster is actually never a good idea,

maybe the neural net's parameters were initialized so that  $Q(s, \text{main})$  is always very low. if that's the case, then the neural networks, because it's trying to pick the action " $a$ " that maximizes  $Q(s, a)$ , it will never try firing the main thruster. ~~is actual~~ Because it never tries firing the main thruster, it will never learn that firing the main thruster is actually sometimes a good idea.

The idea of picking action randomly is called Exploration Step.

in the option (2), the step (1) is called exploitation/greedy step, the step (2) is called exploration step, & you have to balance these two steps, meaning how often you should pick random actions

The option (2) is called Epsilon greedy policy  
E-greedy Policy ( $\epsilon = 0.05$ )

one of the trick that sometimes used it to have a high epsilon at the starting then gradually decrease it, so that overtime you are less likely to take actions randomly.

→ RL algorithms are very sensitive to hyper-parameters, for example if you set the value of epsilon not quite as well, or set other parameters, it may take 10 or a 100 times as long to converge or learn.

# Algorithm refinement: Mini-batch & Soft updates.

→ when the training set size is very large we use batch mini-batch gradient descent,

The idea is to randomly choose few examples, calculate the cost & then using that update the parameter.

{ like if you have let's say 1000 examples you randomly shuffle it into batches of 10 then train the model on all those 10 batches & repeat. }

Soft update →

in the Learning algo just 3 pages before,  
we wrote,

Set  $Q = Q_{\text{new}}$ .

This is an abrupt change to the ~~parameter~~  $Q$ .

So,  $Q = Q_{\text{new}}$  ←  
 $(w, b) \quad (w_{\text{new}}, b_{\text{new}})$

$$\Rightarrow w = 0.01 * w_{\text{new}} + 0.99 * w$$

$$\Rightarrow B = 0.01 * B_{\text{new}} + 0.99 * B, \text{ this will prevent}$$

the abrupt changes, since we are adding a very small amount of  $(w_{\text{new}}, b_{\text{new}})$ , in the using the mini batch update method

## Limitations of RL

- much easier to get to work in a simulation than a real robot
- far fewer applications than supervised & Unsupervised learning.
- But exciting research direction with potential for future directions.